Compression Enables Generalization: Wake-Sleep Cycles for Logic Programming with LLM Integration

Alexander Towell
Southern Illinois University Edwardsville
Edwardsville, IL, USA
lex@metafunctor.com
ORCID: 0000-0001-6443-9897

Hiroshi Fujinoki
Southern Illinois University Edwardsville
Edwardsville, IL, USA
hfujino@siue.edu

***Abstract*—In LLM-augmented rule learning, the operational question is no longer whether a system can discover rules but whether the discovered rules are genuine compression-driven generalizations or patterns the LLM memorized during training. We introduce a test protocol that makes this distinction auditable: fact bases over invented vocabulary the LLM cannot have seen in training, paired with a raw-LLM baseline on the same inputs. Under it, MDL-driven compression of an unannotated knowledge base recovers three rule types with a clean division of labor. Within-predicate generalizations come from symbolic compression: on a synthetic crafting domain it recovers 53% of held-out cases, an LLM-as-hypothesis-generator step raises this to 64 ± 2%, and direct LLM prompting recovers 0%. Recursive transitive closures are the sharp case: a symbolic operation recovers them at 100% recall and 100% precision even on invented vocabulary where the raw LLM recovers 0%, so recursion, the rule type the LLM has most plainly memorized, is recovered without it. Cross-predicate semantic rules are where the LLM is essential. DreamLog, our system, runs wake-sleep cycles inspired by DreamCoder, with each sleep-phase compression verified against positive and negative queries under atomic rollback; on a canonical family tree the full pipeline reaches 80% recall at 100% precision, and it recovers the recursive ancestor closure that the ILP system Popper cannot. These results are consistent with the Solomonoff thesis that shorter programs generalize better, while making the compression-versus-memorization distinction empirically testable.**

***Index Terms*—compression-based learning, logic programming, wake-sleep cycles, knowledge base compression, inductive logic programming, Solomonoff induction, LLM integration**

I. Introduction

Knowledge bases in logic programming accumulate facts over time, yet this accumulation does not constitute learning. A knowledge base with 300 ground facts about a family tree cannot determine whether a newly introduced person is someone's father, even though the concepts `father`, `mother`, and `grandparent` are implicit in the data. The facts grow, but understanding does not emerge.

This paper presents an empirical test of a classical theoretical claim: *compression is learning*. Solomonoff's theory of inductive inference [1] and the closely related Minimum Description Length (MDL) principle [2], [3] assert that the shortest program consistent with observations provides the best predictions. We test this claim in a logic programming setting, where "shorter program" means fewer clauses in a knowledge base (KB), and "better predictions" means the ability to derive facts about entities never seen during training.

We introduce DreamLog, a logic programming system that implements *wake-sleep cycles* inspired by DreamCoder [4]. During wake phases, DreamLog answers queries via SLD resolution, optionally invoking a large language model (LLM) to generate rules for undefined predicates. During sleep phases, eight compression operations transform accumulated facts into general rules. The operations are guided by MDL and verified against a test suite of positive and negative queries, with atomic rollback ensuring behavioral preservation.

This paper makes two interlocked contributions: a test protocol that makes the compression-versus-memorization distinction auditable in LLM-augmented rule learners, and empirical evidence under that protocol that KB compression produces rules generalizing to unseen entities. We instantiate the protocol on two domains:

1) **A novel synthetic domain.** We construct a crafting system with 15 invented materials ("lumite," "vexal," "drossite") that no LLM has encountered in training. Compression discovers concepts such as `master_artisan` and `safe_recipe` purely from structural patterns, achieving 53% recall on unseen entities through symbolic compression alone, and 64 ± 2% with LLM-assisted operations (a raw LLM baseline achieves 0%).
2) **A canonical family tree.** On a 29-person, 4-generation family tree, the full pipeline achieves 80% recall on newly introduced individuals with 100% precision, compressing the KB from 300 to 184 clauses.

The novel domain result is the stronger validation. Because the LLM has never seen the invented terms, it cannot retrieve memorized rules from training data. The symbolic operations discover generalizations through MDL-driven compression of structural regularities in the data, without any labeled training

examples or domain-specific inductive bias.

Our contributions are:

- **A test protocol** for auditing compression-driven generalization in LLM-assisted rule learners: a fact base over invented vocabulary plus a raw-LLM baseline on the same inputs, isolating compression-driven discovery from LLM training-data memorization.
- **A division-of-labor finding** that the protocol makes visible: symbolic MDL compression recovers within-predicate generalizations and recursive transitive closures (the latter at 100% on invented vocabulary where direct LLM prompting recovers 0%), while the LLM is essential only for cross-predicate semantic rules.
- **Operation I**, a symbolic sleep operation that discovers recursive closures (e.g. `ancestor`) by detecting when one relation is the transitive closure of another, needing neither metarules nor labeled examples.
- **A head-to-head comparison** with Popper [5] (Section IV-E): comparable recall on the canonical domain but lower precision, lower recall on the novel-vocabulary domain, and the recursive `ancestor` closure that DreamLog recovers via Operation I but Popper cannot.
- **The wake-sleep architecture**: symbolic compression operations plus LLM-assisted rule discovery, all verified against behavioral test suites with atomic rollback.
- **An ablation and long-lived accumulation study** showing the complementary contributions of symbolic vs. LLM operations and an invest-then-compress-then-saturate dynamics consistent with DreamCoder's convergence.

II. Background and Related Work

A. Compression and Induction

Solomonoff's theory of inductive inference [1] formalizes the intuition that simpler explanations generalize better. The Kolmogorov complexity of a string is the length of the shortest program that produces it; Solomonoff induction predicts future data by weighting hypotheses inversely by their description length. While Kolmogorov complexity is uncomputable, the MDL principle [2], [3] provides a practical approximation: among models consistent with data, prefer the one with the shortest total description (model plus data given model). In our setting, the "model" is the set of rules in a KB, and the "data given model" is the set of remaining ground facts not derivable from those rules.

Modern empirical work supports the compression-prediction equivalence framing on which we rely. Delétang et al. [6] show that large language models are powerful general-purpose compressors and that the compression viewpoint clarifies scaling laws, tokenization, and in-context learning. Lotfi et al. [7] prove PAC-Bayes generalization bounds via parameter compression that are tight enough to explain why overparameterized neural networks generalize. We do not claim our MDL criterion is tight in the same sense (clause count is a coarse proxy for Kolmogorov complexity), but both lines of work ground the use of compression as a learning signal in measured rather than purely axiomatic terms.

B. MDL-Driven Compression of Logic Programs

The idea that compressing a logic program improves what it generalizes predates this work, and predates the LLM era, in two systems we must distinguish ourselves from carefully. Knorf [8] restructures an inductive logic programming knowledge base to minimize its size and redundancy, formulating refactoring as constraint optimization and introducing new predicates through invention during the rewrite. Its central empirical finding is exactly the thesis of this paper, established four years earlier: learning from a refactored (compressed) knowledge base improves an ILP system's predictive accuracy and reduces its learning time, because removing redundancy and factoring out shared structure leaves the knowledge in a more reusable form. Knorf's size-minimizing rewrite with predicate invention is the direct ancestor of our Operations A, B, and D, and its companion notion of *forgetting* background knowledge [9] parallels our usage-based pruning (Operation F). The Apperception Engine [10] makes the same point from the unsupervised direction: it induces Datalog causal theories from raw sensory sequences under a parsimony (cost, i.e. description-length) objective, and is evaluated by how well those theories predict, retrodict, and impute *unseen* sensor readings, outperforming neural baselines. Apperception is the closest classical statement of our central claim that the shortest theory preserving the data generalizes best.

We therefore do not claim that MDL-driven recursion discovery is itself novel; both systems above already perform MDL-style induction of recursive, predicate-inventing logic programs, and our symbolic Operation I (which recovers a recursive transitive-closure rule such as `ancestor` from a flat fact base) sits squarely in their tradition. What neither system can do, because both predate widely available LLMs, is the contrast that isolates this mechanism from memorization. On recursion, the rule type a modern LLM has most plainly absorbed from its training corpus, we run the symbolic compressor and a raw LLM on the *same* task over a deliberately invented vocabulary that cannot appear in any pretraining set (Section IV). The compressor recovers the recursive rule and generalizes to unseen entities at 100%; the raw LLM, denied the lexical cues it would normally pattern-match, recovers 0%. The novel element is not the MDL machinery and not the LLM, but this division of labor made measurable: an invented-vocabulary protocol that attributes the recursion result to compression rather than recall, a distinction Knorf and Apperception had no occasion to draw.

C. DreamCoder and Library Learning

DreamCoder [4], [11] introduced wake-sleep library learning for program synthesis. During wake phases, it solves programming tasks using a library of primitives. During sleep phases, it compresses solved programs to extract reusable abstractions (via anti-unification of lambda terms), which become new library entries. Over iterations, the library converges

to a small set of powerful primitives. DreamLog adapts this architecture to logic programming: our "library entries" are Horn clause rules, our "programs" are KB clauses, and our compression operations (particularly Operations C, D, and E) play the role of DreamCoder's abstraction phase.

Two successors to DreamCoder bear directly on our work. Stitch [12] reformulates the abstraction step as top-down corpus-guided synthesis, achieving 1000–10000× speedup over DreamCoder while preserving library quality. Operations D (predicate invention via skeleton fingerprinting) and E (body pattern extraction) play the role of Stitch for logic programs rather than λ-calculus programs: where Stitch identifies shared λ-term structure across solved tasks, our operations identify shared rule-body structure across the existing KB. LILO [13] extends DreamCoder with LLM-driven documentation and naming of discovered library functions, while keeping the symbolic compression mechanics. LILO is the closest published spirit to DreamLog: both compress a body of structured artifacts and use the LLM as a complement to symbolic discovery. The two systems differ in three ways. First, in what they compress: LILO's library entries are λ-calculus functions (e.g., `(λ x. cons x nil)` as a primitive); DreamLog's are Horn clauses (e.g., `father(X,Y) :- parent(X,Y), male(X)`). Second, in what the LLM contributes: LILO uses the LLM to name and document a library function whose body the symbolic engine has already discovered; DreamLog uses the LLM (Operation G) to *propose* a rule body that the symbolic engine could not have discovered (cross-predicate rules whose body literals span multiple functors, beyond the reach of within-predicate anti-unification). Third, in verification: LILO checks I/O correctness on held-out tasks; DreamLog checks query equivalence via the verification suite (Definition 1). LILO's input is a stream of *tasks with examples*; DreamLog's input is an *unannotated fact base*.

D. Inductive Logic Programming

Inductive Logic Programming (ILP) [14] learns logic programs from labeled positive and negative examples. FOIL [15] uses information gain to grow clause bodies. Progol [16] uses inverse entailment to construct the most specific hypothesis and searches for generalizations. Metagol [17] uses meta-interpretive learning with declarative bias in the form of metarules. ILASP [18] learns Answer Set Programs from partial interpretations. Popper [5] uses hypothesis testing and constraint learning to prune the search space. ∂ILP [19] and Neural Theorem Provers [20] differentiably learn rules from examples. LINC [21] combines LLMs with first-order logic provers for reasoning.

DreamLog differs from ILP in two key respects. First, DreamLog derives its training signal from the KB itself under the closed-world assumption, rather than requiring externally provided labeled examples. All ground facts serve as positive examples, and systematically generated perturbations serve as negative examples. Second, DreamLog's learning signal is compression, not classification accuracy. Rules are accepted only if they reduce description length while preserving the deductive closure of the KB.

E. Anti-Unification

Anti-unification, introduced by Plotkin [22] and Reynolds [23], computes the least general generalization (lgg) of two terms. It is the dual of Robinson's unification [24]: where unification finds the most general term subsuming both inputs, anti-unification finds the most specific term that both inputs are instances of. DreamLog uses anti-unification in Operations C and D to identify shared structure across groups of facts and rules.

F. Predicate Invention

Predicate invention [25], [26] creates new predicates not present in the background knowledge. Metagol [17] invents predicates using metarules. Predicate invention is considered one of the most challenging aspects of ILP [27]. DreamLog's Operation D discovers structurally identical rule sets via skeleton fingerprinting and extracts parameterized predicates using `call/N` dispatch, analogous to lambda abstraction and application.

G. Neural-Symbolic Integration

DeepProbLog [28] integrates neural networks with probabilistic logic programming. Logic Tensor Networks [29] ground logical formulas in real-valued tensors. NeuralLog and related approaches [30], [31] represent relations as learned embeddings. Knowledge graph completion methods (TransE [32], RotatE [33]) learn embedding-based link prediction.

A second cluster combines LLMs with symbolic solvers at inference time. Logic-LM [34] translates natural-language premises into a symbolic form, runs a classical solver, and refines via solver error messages; the LLM acts at parse time and the symbolic engine handles reasoning. LINC [21] follows the same pattern with first-order logic provers. DreamLog locates the LLM at a different stage: not premise translation but rule *learning*, where the LLM (Operation G) proposes candidate clauses that the symbolic evaluator verifies before they enter the KB. This preserves interpretability, since every derived fact retains a human-readable proof trace through Horn clause resolution, while exploiting LLM priors for the cross-predicate patterns that within-predicate anti-unification cannot reach.

Operation G belongs to a fast-growing family in which an LLM proposes hypotheses and a symbolic backend accepts or rejects them, and we position it against that family rather than claiming the pattern as new. The design rationale is established by two ICLR 2024 studies of inductive reasoning. Hypothesis Search [35] prompts an LLM for abstract hypotheses, realizes them as executable programs, and selects by execution on examples; Hypothesis Refinement [36] iterates propose-select-refine and finds that LLMs are excellent hypothesis *proposers* but unreliable rule *appliers*, so that pairing them with a symbolic interpreter that filters the proposed rules is what

yields strong, generalizing results. This proposer-weak-but-verifiable-strong finding is precisely the premise of Operation G: the LLM supplies cross-predicate candidates and the SLD engine, not the LLM, decides what is sound. Two systems apply this premise to logic-program induction specifically. HtT [37] has an LLM induce and verify a reusable rule library and then deduce with it, controlling for memorization through base-randomized arithmetic and held-out CLUTRR kinship splits. The Idiap system [38] couples LLM theory refinement with a formal ILP engine and grades difficulty by rule-dependency structure, benchmarking head-to-head against Popper. DreamLog differs from all four on three axes that we make explicit. First, input modality: HtT, Idiap, and the two ICLR studies consume labeled examples or example-bearing tasks, whereas Operation G consumes an unannotated, accumulating fact base and synthesizes its own positive and negative queries under the closed-world assumption. Second, the acceptance signal: they retain rules by answer correctness, firing frequency, or F1 against examples, whereas Operation G retains a clause only if it shortens the KB's description length while preserving deductive closure, with a closed-world false-positive check and atomic rollback. Third, the verifier: it is a classical SLD resolution engine enforcing query equivalence, not the LLM itself and not an example-graded fitness score. We make no claim of novelty for the propose-then-verify pattern as of 2026; for a current map of this neighborhood under abduction, deduction, and induction we refer the reader to a recent survey [39].

H. Never-Ending Learning

NELL [40] continuously learns beliefs from the web, accumulating an ontology over years. DreamLog shares the goal of continuous knowledge accumulation but operates on a different axis: rather than acquiring new facts from external sources, it discovers structure in existing facts through compression. The two approaches are complementary.

III. System Design

DreamLog is a logic programming system with S-expression syntax that operates in alternating wake and sleep phases. This section describes the architecture and the eight compression operations that constitute the sleep phase.

A. Wake Phase

During the wake phase, DreamLog accepts assertions and queries. Assertions add ground facts (e.g., `(parent alice bob)`) or user-defined rules to the knowledge base. Queries are answered via SLD resolution [41] with negation-as-failure (NAF) [42], supporting:

- **`not/1`**: Negation-as-failure with floundering guard.
- **`call/N`**: Meta-predicate for runtime predicate dispatch.
- **Usage tracking**: Each clause records how often it fires during resolution, providing frequency data for sleep-phase operations.

Algorithm 1 Dream Cycle

Require: Knowledge base K, verification flag v
$K_0 \leftarrow \text{copy}(K)$
$S \leftarrow \text{BuildVerificationSuite}(K)$ {Pos/Neg queries}
$K \leftarrow \text{OpF}(K)$ {Dead clause pruning (wake data)}
$K \leftarrow \text{OpA}(K)$ {Subsumption elimination}
$K \leftarrow \text{OpB}(K)$ {Redundant fact pruning}
$K \leftarrow \text{OpC}(K, S)$ {Fact generalization}
$S \leftarrow \text{ExtendSuite}(S, K)$ {Add rule-derived queries}
$K \leftarrow \text{OpD}(K, S)$ {Predicate invention}
$K \leftarrow \text{OpE}(K, S)$ {Body pattern extraction}
$K \leftarrow \text{OpI}(K, S)$ {Recursive closure discovery}
$K \leftarrow \text{OpG}(K, S)$ {LLM-assisted compression}
$K \leftarrow \text{OpB}(K)$ {Re-prune (post-LLM)}
$K \leftarrow \text{OpH}(K)$ {Lemma caching}
if v **and** $\neg\text{Verify}(K, S)$ **then**
  $K \leftarrow K_0$ {Atomic rollback}
end if
return K, $|K|/|K_0|$

When a query references an undefined predicate, the system optionally invokes an LLM to propose rules defining that predicate. Generated rules undergo structural validation, optional LLM-as-judge verification, and are added to the KB only if they pass all checks. This mechanism transforms undefined predicates from errors into opportunities for knowledge acquisition.

B. Sleep Phase: The Dream Cycle

The sleep phase compresses the KB through eight operations, executed in sequence. Algorithm 1 outlines the overall procedure. Operations that modify the KB (C, D, E, G) perform per-candidate verification against the test suite, accepting each candidate only if it preserves behavioral equivalence. A final pipeline-level verification with atomic rollback provides an additional safety net.

Definition 1 (Verification Suite): Given a KB K, the verification suite $S = (S^+, S^-)$ consists of:

- $S^+ = \{t \mid \text{Fact}(t) \in K\}$: all ground facts.
- S^-: synthetic negative queries formed by substituting atoms into existing fact templates at positions where those atoms do not appear in K.

A compression passes verification if every $q \in S^+$ remains derivable and no $q \in S^-$ becomes derivable.

C. Operation A: Subsumption Elimination

Removes clauses subsumed by more general clauses already in the KB. A rule R_1 subsumes R_2 if there exists a substitution θ such that $R_1\theta \sqsubseteq R_2$ and both have the same body length. Additionally, bodyless rules (rules with empty bodies) subsume matching facts. This removes redundancy introduced by prior operations or user input.

D. Operation B: Redundant Fact Pruning

Removes facts that are derivable from the remaining KB (rules plus other facts). Each fact f is tentatively removed; if f remains derivable via SLD resolution, the removal is permanent. Batch removal is attempted first, with one-at-a-time fallback for mutually dependent facts. This is the primary mechanism by which rule discovery translates into fact removal.

E. Operation C: Fact Generalization with Exceptions

This is the central symbolic compression operation. Given a group of $n \geq 3$ facts sharing a functor and arity, the operation:

1) **Partitions** facts by argument position: for each position p, groups facts that agree on all arguments except position p.
2) **Finds a guard predicate**: a unary predicate g whose extension covers all values appearing at position p in the group (and possibly more).
3) **Computes exceptions**: values in the guard's extension not appearing in the fact group.
4) **Applies MDL**: the generalization is accepted only if $1 + |\text{exceptions}| < n$ (one rule plus exception facts is shorter than the original n facts).

Example 2: Given facts `(master_artisan kael)`, `(master_artisan sera)`, ..., `(master_artisan zara)` covering 7 of 9 artisans, and the guard predicate `artisan/1` covering all 9, Operation C produces:

```
(master_artisan X)
:- (artisan X), (not
(exception_master_artisan_artisan
X))
(exception_master_artisan_artisan
thom)
(exception_master_artisan_artisan
fen)
```

This replaces 7 facts with 1 rule + 2 exception facts = 3 clauses, a net reduction of 4.

F. Operation D: Predicate Invention

Discovers structurally identical rule sets across different predicates and extracts a parameterized "invented" predicate. The procedure:

1) **Extract skeletons**: For each predicate's rule set, compute a *skeleton* that abstracts functor names into roles (SELF for recursive calls, PARAM_i for distinct body functors) while preserving rule structure, arity, and variable connectivity.
2) **Group by skeleton**: Predicates with identical skeletons are structurally equivalent.
3) **Build parameterized predicate**: Create an invented predicate with an additional `call/N` dispatch parameter, and replace each original predicate with a one-line wrapper delegating to the invented predicate.
4) **Apply MDL**: Accept only if $k + m < m \cdot k$, where k is the number of rules per predicate and m is the number of predicates in the group.

This is the logic programming analog of lambda abstraction: the skeleton captures shared computational structure, and `call/N` dispatch plays the role of function application.

G. Operation E: Body Pattern Extraction

Finds common contiguous sub-goal sequences across rule bodies and extracts them as named predicates. Interface variables (those appearing both inside and outside the subsequence) become the extracted predicate's arguments. This discovers shared computational fragments across rules, analogous to common subexpression elimination.

H. Operation F: Dead Clause Pruning

Removes clauses with zero usage after sufficient wake-phase queries. Requires both a minimum query count and that at least 50% of predicates have been exercised. User-provided seed facts are protected from pruning; only clauses added by prior dream cycles (lemmas, LLM-generated rules) are eligible for removal.

I. Operation G: LLM-Assisted Compression

The symbolic operations (A–F) cannot discover rules that cross predicate boundaries (e.g., `father(X,Y) :- parent(X,Y), male(X)`) because they operate within a single functor's fact group. Operation G invokes an LLM to propose such rules.

The pipeline:

1) **Prompt construction**: Sample facts ensuring every predicate is represented (round-robin), include predicate fact counts to guide directionality (specific $\leftarrow$ general).
2) **Response parsing**: Parse JSON-formatted rule proposals, handling common LLM formatting errors.
3) **Cycle filtering**: Reject rules creating cross-functor cycles in the dependency graph via DFS. Self-recursive rules (depth-limited by the evaluator) are permitted.
4) **Helper separation**: Separate helper predicates (new predicates supporting `not/1` patterns) from main rules (deriving existing facts).
5) **Evaluation**: Each main rule must derive ≥ 2 existing facts.
6) **False-positive check**: Enumerate solutions and reject rules deriving ground terms absent from the KB.
7) **Combined verification**: Verify the full set of accepted rules against the verification suite with bounded evaluation to prevent combinatorial explosion.

J. Operation H: Lemma Caching

Adds frequently derived intermediate terms as ground facts for faster resolution. Wake-phase derivation tracking identifies terms computed repeatedly during SLD resolution. Caching these as facts converts multi-step derivations into direct lookups, yielding up to 23.6$\times$ query speedup (Section IV).

K. Operation I: Recursive Closure Discovery

The symbolic operations (A–F) cannot discover recursive rules, and Operation G's prompt does not invite self-reference, so a recursive relation such as `ancestor` is never compressed and its full extension stays stored as ground facts. This is the single largest untapped compression opportunity: the `ancestor` extension is exactly the transitive closure of `parent`, so a two-clause recursive definition can replace the entire stored closure. Operation I is the symbolic route to that recursion.

For each ordered pair of binary predicates (R, B), the operation tests whether R's ground extension equals the irreflexive transitive closure of B's extension. The procedure:

1) **Collect extensions**: gather the set of ground argument pairs for every binary predicate (over atom-valued arguments).
2) **Match against closure**: for each candidate base B with at least `min_base_facts` pairs, compute $\mathrm{TC}(B)$ by graph reachability and test $\mathrm{ext}(R) = \mathrm{TC}(B)$.
3) **Synthesize the pair**: on a match, build the base rule `R(X,Y) :- B(X,Y)` and the right-recursive rule `R(X,Z) :- B(X,Y), R(Y,Z).`
4) **Verify and replace**: verify the pair against the verification suite with the bounded evaluator, then replace R's facts with the two rules.

The MDL gain is large and unambiguous: two clauses replace the N stored closure facts, so acceptance turns entirely on verification. Termination is handled by construction. Left-recursion is excluded as a safeguard, and the bounded evaluator caps total resolution calls, so a candidate that does not terminate or over-runs the budget surfaces as a verification *failure* (the closure facts simply stop being derivable) rather than an exception. Operation I runs in the symbolic phase, after Operation E and before Operation G, so the symbolic-only and full pipelines differ only by Operation G. It is gated by a flag and off by default, preserving the zero-drift behavior of the standard pipeline. This version handles the transitive closure of a single binary base relation and right-recursion only; closure path length is bounded by the evaluator's recursion-depth limit.

Example 3: Given `parent` facts forming a chain (`(parent a b), (parent b c), (parent c d), ...`) and the materialized closure stored as `ancestor` facts, Operation I detects that the `ancestor` extension equals $\mathrm{TC}($`parent`$)$ and emits:

```
(ancestor X Y) :- (parent X Y)
(ancestor X Z) :- (parent X Y),
(ancestor Y Z)
```

After verification accepts the pair, the `ancestor` facts become redundant (each is re-derivable through the recursion), and Operation B prunes every one of them.

L. Behavioral Preservation

All operations share a common verification protocol. Before any modification, the system:

1) Takes a snapshot of the KB.
2) Constructs (or extends) the verification suite.
3) Applies the proposed compression.
4) Verifies that all positive queries remain derivable and no negative query becomes derivable.
5) Rolls back to the snapshot if verification fails.

This ensures that the compressed KB is *behaviorally equivalent* to the original on all tested queries. Experiment EX02 confirmed zero semantic drift across 200 held-out queries over multiple dream cycles.

IV. Experiments

We organize the experiments by the rule type each mechanism recovers, the division of labor the test protocol makes visible: within-predicate generalizations from symbolic compression (Section IV-B), recursive transitive closures from a symbolic operation (Section IV-C), and cross-predicate semantic rules from LLM-proposed, symbolically-verified hypotheses (Section IV-D). A head-to-head with the modern ILP system Popper (Section IV-E) and a set of supporting studies (long-lived accumulation, ablation, scaling, noise tolerance, and query speedup) follow. All experiments use Anthropic Claude Haiku 4.5 for the LLM-assisted operations, with total LLM cost well under one dollar across all experiments.

A. Experimental Setup

All experiments run on DreamLog's Python implementation with SLD resolution, NAF, and `call/N`. The LLM provider for Operation G is Anthropic Claude Haiku 4.5 ($0.25/M input tokens) via API. Each dream cycle invokes the LLM at most twice (once for rule proposal in Operation G, once for predicate naming). Verification suites are constructed automatically from KB state. All compression operations use default parameters: minimum group size 3, shared structure threshold 0.1, maximum 200 prompt facts.

B. EX25b: Generalization on a Novel Domain

1) Motivation: The canonical test for compression-driven generalization (a family tree) is problematic: the LLM has likely seen `father(X,Y) :- parent(X,Y), male(X)` thousands of times during training. We cannot distinguish compression-driven discovery from training data recall. This experiment uses a synthetic domain with invented terms the LLM has never encountered.

2) Domain: An alchemical crafting workshop with 15 materials having invented names (lumite, vexal, drossite, pyreth, quarl, noctite, aerine, sylphex, thalline, morven, zephore, ethylis, fenrik, glacine, bramith) and properties (phase, material class, hazard status). The domain includes 16 recipes combining materials into products, 9 artisans with skills, and 5 skill requirements. Total: 112 base facts and 61 derived facts across 6 predicates (`hazardous_recipe`, `safe_recipe`, `same_phase_recipe`, `metallic_alloy`, `can_craft`, `master_artisan`).

TABLE I
GENERALIZATION ON THE NOVEL CRAFTING DOMAIN (EX25B). 33 CHECKS (19 POSITIVE, 14 NEGATIVE) ON UNSEEN ENTITIES. LLM CONDITIONS: MEAN ± STD OVER 5 RUNS.

Condition	Rules	Acc	Prec	Recall
No dream	0	42%	—	0%
Raw LLM	0	42%	—	0%
Symbolic	5	52%	59%	53%
Full	20	58 ± 1%	64 ± 1%	64 ± 2%

3) Protocol: Four conditions: (1) *no dream*: baseline KB with no compression; (2) *symbolic*: dream cycle with Operations A–F only (no LLM); (3) *full pipeline*: dream cycle with all operations including LLM-assisted Operation G; (4) *raw LLM*: prompt Haiku directly with all KB facts and new-entity queries (no compression pipeline). After dreaming, 6 new materials, 8 new recipes, and 4 new artisans are introduced with base facts only. 33 checks (19 positive, 14 negative) test whether derived predicates are correctly inferred for the unseen entities. LLM-dependent conditions are run 5 times; we report mean ± standard deviation.

4) Results: Table I shows the results.

The *raw LLM* baseline achieves 0% recall: Haiku answers "NO" to every query about invented terms, confirming that direct LLM prompting contributes nothing on novel domains. The compression pipeline is genuinely necessary.

Symbolic compression alone discovers 5 rules and achieves 53% recall. The key discoveries are:

- `master_artisan(X) :- artisan(X), not(exception...)`: generalizes from the pattern that 7 of 9 artisans have 2+ skills.
- `safe_recipe(X) :- product(X), not(exception...)`: generalizes from the pattern that most recipes use non-hazardous materials.

These are discovered by Operation C through MDL-driven fact generalization. No domain knowledge is used; the system finds that "most artisans are master artisans" and "most recipes are safe" purely from the statistical structure of the data.

The full pipeline (mean of 5 runs) achieves $64 \pm 2\%$ recall with 20 discovered rules. Operation G adds cross-predicate rules including `hazardous_recipe` (via material hazard properties) and concepts like `volatile_material` and `metallic_material`. The low variance ($\pm 2\%$) confirms that results are stable across LLM generations.

Seven false positives occur from `safe_recipe` and `same_phase_recipe` generalizations applying to exception cases (e.g., a liquid-solid mix classified as same-phase). This illustrates the inherent risk of inductive generalization.

The undiscovered predicate is `can_craft(Artisan, Product)`, which requires a 3-hop join (artisan → skill → requires_skill → recipe_type). This exceeds the current operations' reach.

C. EX27: Recursive Closure Discovery

1) Motivation: Recursive rules, and the transitive closure in particular, are the canonical hard case for inductive logic programming. Metagol requires hand-supplied metarules to admit recursion, and Popper requires labeled positive and negative examples plus a recursion-enabling bias. EX26 had to exclude the recursive `ancestor` target entirely: neither DreamLog (whose Operations A–H find patterns within and across flat predicates but not recursive definitions) nor Popper (which hung in its `clingo` backend past the Python-level timeout) recovered it. Recursion is therefore the single largest gap in the preceding experiments. It is also the rule type the LLM has most plainly memorized: `ancestor(X,Z) :- parent(X,Y), ancestor(Y,Z)` appears verbatim across countless Prolog tutorials. This makes it the sharpest available test of whether DreamLog's discovery is compression-driven or merely LLM recall. We add a symbolic operation, Operation I, that detects when a binary predicate `R` is the exact transitive closure of another binary predicate `B` and synthesizes the right-recursive definition, and we ask whether that discovery survives the invented-vocabulary protocol of EX25b.

2) Domains: Two transitive-closure domains, one canonical and one invented, mirroring the EX25/EX25b pairing. The canonical domain is `ancestor` as the transitive closure of `parent` over real first names (`alice`, `bob`, `carol`, ...), the relation and vocabulary the LLM has most certainly seen in training. The invented domain is `flux_reaches` as the transitive closure of `flux_links` over invented node names (`qux`, `vor`, `zane`, `plix`, ...) that the LLM has not encountered as graph nodes. Each training graph is an acyclic forward chain over 8 nodes with a few extra forward edges; the 28 training derived facts are the *exact* closure of the 8 training base edges, so Operation I's exact-match gate can fire.

3) Protocol: We use the new-entity protocol of EX25/EX25b rather than a holdout split, because holdout would leave the training closure incomplete and the exact-match gate would never trigger. We dream on the full training closure, then introduce 4 new entities with base edges only (extending the forward order so the combined graph stays acyclic), and check whether the discovered rule derives their closure pairs. The 76 new-entity checks per domain (38 positive, 38 closed-world-balanced negative) test reachability for pairs that touch a new node. Four conditions, seed 42: (1) *no dream*: baseline KB with no compression; (2) *symbolic*: Operation I only, no LLM; (3) *full*: Operation I followed by a recursion-aware Operation G; (4) *raw LLM*: Haiku used directly as an inference engine, answering yes/no on each held-out query with the training facts in context. Verification is closed-world with the bounded evaluator; path lengths stay well under the evaluator's recursion-depth ceiling.

4) Results: Table II shows the results.

Symbolic compression alone, with no LLM, recovers the recursive definition on both domains and classifies every new-entity check correctly: 100% recall and 100% precision. On the invented `flux` domain it discovers

- `(flux_reaches X Y) :- (flux_links X Y)` and

TABLE II
RECURSIVE CLOSURE DISCOVERY (EX27). NEW-ENTITY PROTOCOL, SEED 42, 76 CHECKS PER DOMAIN (38 POSITIVE, 38 NEGATIVE). SYMBOLIC OPERATION I ALONE REACHES 100% RECALL AND 100% PRECISION ON BOTH DOMAINS, INCLUDING THE INVENTED ONE WHERE THE RAW LLM RECOVERS NOTHING.

Domain	Condition	Recall	Prec	Acc	Rules
Flux	No dream	0%	—	50%	0
(invented)	Raw LLM	0%	—	50%	0
	Symbolic	100%	100%	100%	2
	Full	100%	100%	100%	2
Family	No dream	0%	—	50%	0
(ancestor)	Raw LLM	2.6%	100%	51%	0
	Symbolic	100%	100%	100%	2
	Full	100%	100%	100%	4

- `(flux_reaches X Z) :- (flux_links X Y), (flux_reaches Y Z),`

the base and right-recursive clauses of the transitive closure, and compresses the closure to a ratio of 0.278 (the 28 derived facts collapse into 2 rules). The canonical `ancestor` domain yields the structurally identical pair over `parent`.

The *raw LLM* baseline, by contrast, recovers essentially nothing: 0% recall (0 of 38) on the invented vocabulary and 2.6% recall (1 of 38) on real names. Even though recursion over `parent` is the rule type the LLM has most obviously memorized, using the model directly as an inference engine over held-out queries fails. This is the decisive contrast: because the raw LLM gets 0% on `flux_reaches`, the 100% symbolic result on that domain cannot be memorization. Operation I recovers an unbounded relation from its finite extension purely from MDL-driven structure, with no prior knowledge of the vocabulary.

We are explicit that the *full* pipeline adds nothing here. Operation I runs before Operation G, so by the time the LLM is consulted the recursive rule is already in place and the closure is already compressed; the LLM's recursion proposal is redundant on a clean closure. On `family` the recursion-aware Operation G proposed 2 additional rules (duplicating the clauses Operation I had already synthesized), and recall, precision, and the compression ratio were unchanged; on `flux` it proposed none. The *full* row therefore equals the *symbolic* row, and the LLM should not be credited with the recovery. The LLM cost across the raw-LLM and full conditions was $0.28 (154 Haiku calls), spent almost entirely on the raw-LLM baseline that failed.

5) Interpretation: This is the sharp instance of the division of labor that organizes this section. Recursive closures are a clean *symbolic* capability: Operation I recovers them on vocabulary the LLM never saw, while the LLM, used directly, recovers them on neither the familiar nor the unfamiliar domain. Read together with the cross-predicate experiments, where the LLM is essential because symbolic operations cannot cross functor boundaries, the picture is that the two mechanisms cover disjoint rule types. The result also updates the EX26 framing: where that comparison recorded that neither DreamLog nor Popper recovers `ancestor`, DreamLog now does, via a symbolic operation that needs no labels and no metarules.

TABLE III
GENERALIZATION ON THE CANONICAL FAMILY TREE (EX25). THE FULL PIPELINE ACHIEVES 80% RECALL WITH 100% PRECISION.

Condition	Rules	Acc	Prec	Recall	KB Size
No dream	0	29%	—	0%	300
Symbolic	1	29%	—	0%	—
Full	7	86%	100%	80%	184

We state the boundary plainly. These results are on controlled synthetic closures: exact transitive closure, right-recursion only, and small acyclic graphs. Operation I's exact-match gate is closed-world, so a holdout-style partial closure (where some training closure pairs are withheld) would not trigger it; relaxing the gate to a subset match under an open-world assumption is left to a follow-on data-density study. Operation I is off by default and flag-gated, preserving the zero-drift guarantee of the production pipeline.

D. EX25: Generalization on a Canonical Domain

1) Domain: A 4-generation family tree with 29 people, 96 base facts (`person`, `male`/`female`, `parent`), and 204 derived facts across 7 predicates (`father`, `mother`, `grandparent`, `grandfather`, `grandmother`, `great_grandparent`, `ancestor`).

2) Protocol: Same conditions as EX25b, including the raw LLM baseline. After dreaming, 6 new people are introduced with base facts only, and 14 checks (10 positive, 4 negative) test derived relationships.

3) Results: Table III shows the results.

The full pipeline discovers 7 rules: `father` ← `parent` + `male`, `mother` ← `parent` + `female`, `grandparent` ← `parent` ∘ `parent`, `grandfather` ← `grandparent` + `male`, `grandmother` ← `grandparent` + `female`, and variants. The KB compresses from 300 to 184 clauses (ratio 0.613). Precision is 100%: no false positives.

The sole unrecovered predicate is `ancestor`, which requires a recursive rule (`ancestor(X,Y) :- parent(X,Y)` and `ancestor(X,Z) :- parent(X,Y), ancestor(Y,Z)`). The current LLM prompt does not propose recursive rules, and symbolic operations cannot discover them from flat fact patterns.

Symbolic compression discovers only one entity-specific rule (e.g., "albert is an ancestor of everyone") rather than the general concept. This highlights the critical role of the LLM: symbolic operations find patterns *within* a predicate, while the LLM discovers relationships *across* predicates.

E. EX26: Popper ILP Baseline

1) Motivation: DreamLog and Popper [5] occupy different points in the rule-learning design space. Popper is supervised (it requires positive and negative examples plus mode declarations) and uses a constraint-driven search; DreamLog

is unsupervised (it consumes an unannotated fact base) and uses a compression-driven search. A naive head-to-head would advantage one system or the other depending on how labels are supplied. We construct an apples-to-apples comparison that gives each system the natural form of its required inputs and evaluates both on the same held-out facts.

2) Protocol: For each target predicate, the same EX25/EX25b fact base is converted to Popper input format: `bk.pl` contains the base facts plus the other targets' positives; `exs.pl` contains the held-in derived facts of the target predicate as E^+ and a closed-world random sample (with $|E^-| = |E^+|$, drawn with the same RNG seed as the data split) as E^-; `bias.pl` contains mode declarations from the natural schema. Mode declarations are emitted programmatically from a fixed template embedded in the experiment script, committed to version control before the first result row was recorded; no `bias.pl` is hand-edited after a result is observed. Per-cell timeout is 300 seconds; failures (timeout, parse error, crash) are recorded as zero rules. Recorded Popper runtimes were sub-second for every target on every seed (maximum 1.12 s), well below the budget. We exclude the recursive `ancestor` target: Popper's Python-level timeout does not interrupt its `clingo` C-extension on this configuration, leaving the call unbounded in practice, and EX25 also does not recover `ancestor`, so both systems get a pass on the recursive case here (we revisit recursion in Section IV-C, where Operation I recovers it). Three random seeds (42, 43, 44) per (system, domain) cell, yielding 24 cells total (4 systems $\times$ 2 domains $\times$ 3 seeds). Total wall time was approximately four minutes; LLM cost $0.03 across the six full-pipeline cells.

3) Results: Table IV reports recall, precision, accuracy, and rule count for each cell, averaged over the three seeds. Per-seed values were within 1 percentage point of the cell mean for all reported conditions: Popper is deterministic given a fixed seed (negative samples and search order are seed-determined), and DreamLog full pipeline showed only minor rule-count variance (21–24 on crafting) with identical recall and precision across seeds. The $n = 3$ sample is too small for informative bootstrap confidence intervals, and the reported ranges reflect the full seed-to-seed spread. The 11-percentage-point gap between Popper and DreamLog symbolic on crafting holds at every individual seed; the per-seed Popper crafting recall is 42.4% on all three.

On the family domain, Popper matches DreamLog's full-pipeline recall (both 80%) but trails on precision by 20–33 percentage points. The rules tell the story: Popper discovers `father(X,Y) :- parent(X,Y)` and `mother(X,Y) :- parent(X,Y)` (and analogous over-general grandfather/grandmother rules) because random closed-world negatives do not include "parent but female" near-misses that would force the gender constraint. The two correct rules Popper discovers are `grandparent` and `great_grandparent`, where no auxiliary predicate is needed.

On the crafting domain, the result inverts. Popper's recall (42%) is *below* DreamLog's symbolic-only baseline (53%) by 11 percentage points, and well below the full pipeline (63%). Both Popper and DreamLog-symbolic are pure symbolic learners with no LLM, so the gap argues that DreamLog's MDL-guided fact generalization (Operation C) and skeleton-based abstraction (Operations D, E) have an inductive bias that random-negative ILP does not naturally express. Popper compensates with higher precision (80% vs. DreamLog full's 63%), producing four conservative rules rather than 20+ exploratory ones.

TABLE IV
EX26 POPPER BASELINE. APPLES-TO-APPLES COMPARISON ON THE SAME EX25 FAMILY AND EX25B CRAFTING DOMAINS, THREE SEEDS PER CELL. DREAMLOG CONDITIONS MATCH EX25/EX25B NUMBERS WITHIN ROUNDING; POPPER IS THE NEW DATA POINT.

Domain	System	Recall	Prec	Acc	Rules
Family	No dream	0%	—	29%	0
	Symbolic	0%	—	29%	1
	Full	80%	100%	86%	7
	Popper	80%	67–80%	57–71%	6
Crafting	No dream	0%	—	42%	0
	Symbolic	53%	59%	52%	5
	Full	63%	63%	58%	21–24
	Popper	42%	80%	61%	4

4) Interpretation: We do not read these numbers as "DreamLog beats Popper" or vice versa. The two systems answer different signals. The findings characterize the methodological tradeoff cleanly. On family, the closed-world random-negative protocol is too easy for Popper because the universe of false "father" tuples is dominated by clearly-impossible pairs (e.g., a child as parent of their grandparent); a curated set of near-miss negatives would close the gap. On crafting, hand-curating equivalent near-misses is not natural because the invented vocabulary does not come with a pre-existing intuition about what should be excluded. DreamLog's contribution is operational rather than competitive: when only an unannotated fact base is available, LLM-supplied semantic priors (Operation G) and MDL-guided structural generalization (Operations C, D, E) substitute for the curated supervision Popper relies on.

F. EX25c: Holdout Sweep and the Open-World Mode

1) Motivation: The new-entity tests (EX25, EX25b) introduce entities with no prior facts at all. A complementary protocol, common in inductive logic programming, holds out a fraction of derived facts and asks whether the system can recover them after compressing the remainder. This sweep characterizes the data density threshold for compression-driven generalization on the family tree domain.

2) Closed-World versus Open-World Verification: Operation G's verification pipeline includes a false-positive check: any rule that derives a ground term absent from the KB is rejected. This is correct for the standard compression setting (a complete KB should not be augmented with novel facts) but

TABLE V
HOLDOUT SWEEP ON FAMILY TREE (EX25C). 3 SEEDS PER RATIO. CLOSED-WORLD RECOVERY IS 0% AT EVERY RATIO (THE FP CHECK REJECTS RULES THAT DERIVE ANY HELD-OUT FACT). OPEN-WORLD RECOVERY FOLLOWS A CLEAR CURVE.

Holdout	Closed-world	Open-world	Compression
5%	0%	100%	0.405
10%	0%	100%	0.399
20%	0%	100%	0.415
30%	0%	$85 \pm 21\%$	0.507
50%	0%	$72 \pm 20\%$	0.660

pathological for holdout evaluation, where the goal is precisely to recover absent facts.

We introduce an *open-world mode* that disables this check. In this mode, Op G accepts rules that cover at least 2 existing facts and pass the verification suite, regardless of whether the rule also derives facts not in the KB. The closed-world default preserves the strong safety guarantee for normal use; open-world mode is enabled explicitly during holdout evaluation.

3) Protocol: For each ratio $r \in \{0.05, 0.10, 0.20, 0.30, 0.50\}$, we randomly hold out r of each predicate's derived facts (stratified by predicate). The remaining facts plus all base facts form the training KB. We dream on the training KB (full pipeline, open-world mode) and measure recovery: the fraction of held-out facts derivable from the compressed KB. Three random seeds per ratio.

4) Results: Table V shows the recovery curve.

Three observations:

Open-world recovers all held-out facts at low ratios. At 5–20% holdout, recovery is 100%: the discovered rules (`father` ← `parent` + `male`, etc.) generalize perfectly to the held-out facts. The KB is compressed from 290 to 118 clauses (ratio 0.40), an even stronger compression than the no-holdout setting in EX25 (0.61), because the smaller KB has less verification overhead.

The data density threshold appears at 30%. Recovery drops to $85 \pm 21\%$ at 30% holdout and $72 \pm 20\%$ at 50% holdout. The high variance reflects which facts get held out: some random splits leave enough facts per predicate for Op G to propose a rule that derives at least 2 existing facts (Op G's acceptance gate); others do not. On the family tree, the cross-predicate rules that drive recovery (`father` ← `parent` + `male`, etc.) come from Op G rather than Op C, so the relevant constraint is Op G's minimum-coverage gate, not Op C's `min_group_size`.

No false positives in either mode. Even with the FP check disabled, the rules discovered by Op G do not derive incorrect facts. This is because the rules themselves (`father=parent+male`) are sound; the closed-world FP check was rejecting them not for being wrong but for deriving facts that were intentionally absent.

The closed-world default remains the right choice for production use (strong safety guarantee, zero semantic drift). Open-world mode is an explicit opt-in for evaluation against ILP-style benchmarks where the closed-world assumption is not part of the protocol.

TABLE VI
DREAM CYCLE PROGRESSION IN THE 25-SESSION RESEARCH LAB (EX24).

Cycle	Session	Before	After	Key Discoveries
1	S05	94	91	2 symbolic rules
2	S10	179	195	17 LLM cross-domain rules
3	S15	278	275	2 symbolic rules
4	S20	—	—	Saturation (minor pruning)
5	S25	—	—	No change

G. EX24: Long-Lived Knowledge Accumulation

1) Motivation: Real systems accumulate knowledge incrementally over sessions. We test whether DreamLog exhibits meaningful learning dynamics over an extended interaction.

2) Protocol: A simulated research lab scenario: "Dr. Chen" builds institutional knowledge across 25 sessions spanning two simulated years. Seven domains: people, publications, teaching, grants, infrastructure, collaborations, and service. Sessions vary from 5 to 30 assertions. Session 10 includes corrections (role changes). Session 17 includes user-provided rules with NAF helper predicates. Dream cycles occur every 5 sessions.

3) Results: The final KB contains 451 clauses (422 facts, 29 rules) from 443 total assertions. Table VI shows the dream cycle progression.

Three phases are visible:

1) **Investment** (Dream 2): The LLM discovers 17 cross-domain rules including `grant_funded_work`, `cross_institution_collaboration` (a 4-body rule with NAF), and `mentorship_chain`. The KB *expands* from 179 to 195 (ratio 1.09), investing in structural rules.
2) **Compression** (Dream 3): Symbolic operations capitalize on accumulated rules, achieving net compression.
3) **Saturation** (Dreams 4–5): No new patterns remain. The KB is fully compressed.

This invest-then-compress-then-saturate dynamic matches DreamCoder's convergence behavior [4]. The 29 discovered rules come from three sources: 4 symbolic (Operation C), 17 LLM (Operation G), and 8 user-provided. All 13 correctness checks pass, including transitive citation chains, NAF-based active membership, and negative examples. Total LLM cost: $0.023.

H. Ablation Analysis

Table VII summarizes the ablation from Experiment EX08 on a 119-clause KB with diverse pattern types.

Operation C is the workhorse, contributing 85% of compression on this KB. Operation E *increases* clause count (it adds a clause for the extracted predicate) but improves structural clarity, which may benefit future compression cycles. Operations A and F contribute on KBs with subsumed or dead

TABLE VII
LEAVE-ONE-OUT ABLATION ON A 119-CLAUSE KB (EX08). PERCENTAGES ARE NOT ADDITIVE: EACH ROW SHOWS THE COMPRESSION LOST WHEN THAT OPERATION ALONE IS DISABLED.

Operation	Contribution	Role
C: Generalization	85% (11 clauses)	Primary compressor
B: Fact pruning	15% (2 clauses)	Post-rule cleanup
D: Predicate invention	15% (2 clauses)	Structural abstraction
E: Body extraction	−15% (+2 clauses)	Adds structural clarity
A, F, H	0%	Situational

clauses, and Operation H contributes to query performance rather than KB size.

The generalization experiments (Tables I and III) provide a complementary ablation: symbolic operations (C primarily) discover entity-level generalizations ("most artisans are masters"), while Operation G discovers cross-predicate concepts ("father = male parent"). Neither alone achieves the full pipeline's performance.

I. Additional Results

1) Convergence (EX01): The dream cycle converges in exactly 2 iterations across all tested KBs: cycle 1 compresses, cycle 2 confirms no further opportunities exist. The cycle is a monotone operator on KB size that converges quickly in practice.

2) Semantic Drift (EX02): Zero drift across 200 held-out queries over all dream cycles. The verification suite with positive and negative queries prevents any behavioral change. This is a strong safety guarantee for production use.

3) DreamCoder Benchmark (EX03): On list-processing programs (member, append, reverse, length, map, filter), the system discovers `forall/2`: a parameterized predicate equivalent to DreamCoder's "every" combinator. This validates the architectural analogy between the two systems.

4) Scaling (EX09): Compression ratio is stable at 0.88 across KBs from 69 to 1019 clauses (10 to 200 entities). Throughput degrades from 75 clauses/s to 12 clauses/s due to quadratic verification cost.

5) Noise Tolerance (EX11): Robust to 0–20% noise (incorrect facts left as isolated facts, not generalized). At 30%+ noise, Operation C generalizes erroneous patterns that share sufficient structural regularity.

6) Query Speedup (EX10): After dreaming, query time drops from 25.0ms to 1.1ms (23.6× speedup). Operation H caches 15 frequently derived lemma facts, converting multi-step recursive resolutions into direct lookups.

V. DISCUSSION

A. What Symbolic vs. LLM Operations Contribute

The experiments reveal a clean division of labor. Symbolic operations (particularly Operation C) discover generalizations *within* a predicate: "most artisans are master artisans," "most recipes are safe." These are statistical regularities visible from the distribution of ground facts. The LLM (Operation G) discovers relationships *across* predicates: "father = parent who is male," "hazardous recipe = recipe using a hazardous material." These require understanding how distinct predicates relate semantically.

On the novel crafting domain, symbolic compression achieves 53% recall while the full pipeline reaches 64%. On the canonical family tree, symbolic compression achieves 0% recall on the full family tree (Operations A through F, without Operation I), because its generalization opportunities are either cross-predicate semantic (`father`, requiring Operation G) or recursive (`ancestor`, requiring Operation I; Section IV-C), while the full pipeline achieves 80%. Critically, a raw LLM baseline (prompting Haiku directly with all facts) achieves 0% on both domains, confirming that the compression pipeline provides the generalization capability, not the LLM alone. This asymmetry reflects the nature of the domains: the crafting domain has within-predicate regularities (most artisans are masters) while the family domain's generalization opportunities are entirely cross-predicate.

B. The Novel Domain Isolates Compression from LLM Memorization

The crafting domain result provides stronger evidence for compression-driven generalization than the family tree. When the LLM proposes `father(X,Y) :- parent(X,Y), male(X)` for a family KB, it may be recalling this rule from training data rather than discovering it from structural patterns. The invented terms in the crafting domain (lumite, vexal, drossite) eliminate this confound. That symbolic compression alone achieves 53% recall on invented terms, while the raw LLM achieves 0%, demonstrates genuine MDL-driven concept discovery independent of any LLM prior knowledge.

The family tree remains useful as a benchmark because its higher recall (80%) demonstrates the ceiling achievable when the LLM can draw on domain familiarity. The two domains together bracket the system's capability: novel domains test pure compression, canonical domains test the LLM integration.

C. Limitations

1) Multi-hop rules: The system cannot discover rules requiring 3+ hop joins. `can_craft(Artisan, Product)` requires chaining artisan → skill → requires_skill → recipe_type, which exceeds the body length of rules currently discoverable by either symbolic or LLM operations.

2) Recursion scope: Operation I discovers recursive transitive closures, but only under specific conditions: the target relation must be the *exact* closure of a single binary base relation (a closed-world, exact-match gate, so holdout-style partial closures do not trigger it), the recursion must be right-recursive, and closure path length is bounded by the evaluator's recursion-depth limit. Mutual recursion, accumulator patterns, and list recursion are not addressed. Relaxing the exact-match gate to a subset match under an open-world assumption, for a data-density study of partial closures, is left to future work.

3) Minimum data threshold: The holdout sweep (EX25c) shows recovery degrades sharply once more than 30% of facts per predicate are held out. Op G accepts a proposed

rule only if it derives at least 2 existing facts, so rules become harder to discover as the remaining fact count per predicate shrinks. Op C's generalizations also weaken (its default `min_group_size` is 3), but on the family tree the recovery curve is driven by Op G's cross-predicate rules. Compression-driven learning needs sufficient data density per predicate to activate; the threshold is roughly 7–10 facts per predicate for the family tree domain.

4) Scaling: Verification cost is quadratic in KB size due to exhaustive query checking. KBs beyond 1000 clauses would benefit from sampled verification or cached resolution.

5) LLM dependence for cross-predicate semantic rules: The family tree ablation shows that symbolic operations alone achieve 0% generalization recall when all opportunities are cross-predicate *semantic* rules (e.g. `father` = male parent). The system relies on the LLM for this critical capability, introducing dependence on LLM quality and availability. This dependence is specific to cross-predicate semantic rules: recursive cross-predicate rules such as `ancestor` are recovered symbolically by Operation I (Section IV-C), with no LLM.

D. Relation to Inductive Logic Programming

DreamLog occupies a different point in the rule learning design space than classical ILP systems. We briefly contrast with three representative approaches, clarifying what DreamLog does and does not claim.

a) Popper [5]: treats rule learning as a constraint satisfaction problem. Given explicit positive and negative examples, Popper searches for a hypothesis that covers all positives and excludes all negatives, using failures on counter-examples as constraints to prune the search space. The learning signal is classification accuracy on labeled data. DreamLog's signal is instead compression: rules are accepted when they reduce description length while preserving deductive closure. Section IV-E (EX26) reports the apples-to-apples comparison: Popper supplied with the same KB plus closed-world-sampled negatives matches DreamLog's full-pipeline recall on the family domain (both 80%) but trails by 20–33 precision points (over-general rules missing gender constraints); on the novel crafting domain Popper's recall (42%) falls below DreamLog's symbolic-only baseline (53%). The novel-domain result is the more discriminating one: both systems are pure symbolic learners there, and the gap argues that DreamLog's MDL-guided structural generalization has an inductive bias random-negative ILP does not naturally express. Popper's constraint-driven search and DreamLog's compression-driven search remain complementary signals that could in principle be combined. EX26 had to exclude the recursive `ancestor` target because neither system recovered it; with Operation I (Section IV-C) DreamLog now does, while Popper still cannot complete the recursive case within its `clingo` backend. On the recursive rule type, the two systems are therefore no longer at parity.

b) Metagol [17]: uses meta-interpretive learning with user-provided metarules as declarative bias. The metarules constrain the hypothesis space and make predicate invention tractable. DreamLog's Operation D invents predicates via skeleton fingerprinting without metarules, relying instead on structural equivalence across rule sets already present in the KB. Metagol can solve problems DreamLog cannot (e.g., learning recursive predicates from a few examples when the metarule "closure" is provided), and DreamLog can solve problems Metagol cannot (e.g., discovering that seven of nine artisans share a pattern without needing anyone to write a metarule). Again, the signals are different rather than competitive.

c) LINC [21]: uses an LLM as a translator: natural language premises are converted to first-order logic, then a classical theorem prover decides entailment. LINC's LLM operates at parse time, not inference time. DreamLog uses an LLM differently: Operation G treats the LLM as a *hypothesis generator* for compression candidates, which are then verified by a symbolic evaluator. Both approaches preserve symbolic soundness by keeping the reasoning engine classical, but they apply the LLM at different stages (input formalization vs. rule proposal). LINC does not learn rules; it reasons over given ones. The two systems address different problems.

In summary, we do not claim that DreamLog is a better ILP system than Popper or Metagol. We claim that compression-driven learning from unannotated fact bases is a distinct capability, that it produces rules which generalize to unseen entities, and that combining symbolic compression with LLM-assisted rule proposal yields complementary benefits. EX26 provides a direct head-to-head against Popper; extending the comparison to canonical ILP benchmarks (Michalski trains [14], Bongard problems, NELL [40]) remains future work and would require accepting explicit positive/negative example sets alongside fact bases.

E. Relation to Solomonoff Induction

Our results are consistent with the Solomonoff thesis. The compressed KBs generalize better than the uncompressed originals. However, our MDL criterion (clause count) is a coarse proxy for Kolmogorov complexity, and two domains do not establish a predictive relationship between compression ratio and generalization recall. A more faithful implementation would account for clause complexity (number of variables, body length) rather than treating all clauses as unit cost. We leave this refinement to future work.

VI. Conclusion

We have introduced a test protocol for auditing compression-driven generalization in LLM-assisted rule learners and a wake-sleep logic-programming system, DreamLog, that demonstrates the kind of generalization the protocol is built to detect. The methodological contribution is the protocol: by pairing a fact base over invented vocabulary with a raw-LLM baseline on the same inputs, the protocol separates genuine compression-driven discovery from LLM training-data recall. The empirical contribution is the gap that opens under it. On a novel crafting domain, raw LLM prompting recovers 0% of held-out generalizations, symbolic

compression alone recovers 53%, and the full pipeline reaches $64 \pm 2\%$; on a canonical family tree, the full pipeline reaches 80% recall with 100% precision. A head-to-head comparison with Popper, the modern reference ILP system, places DreamLog's compression-driven approach relative to constraint-driven ILP: comparable recall on the canonical domain but lower precision, and lower recall on the novel-vocabulary domain because random closed-world negatives do not supply the inductive bias DreamLog's MDL-guided structural operations encode.

The system exhibits meaningful learning dynamics over extended interactions: an invest-then-compress-then-saturate pattern consistent with DreamCoder's convergence behavior, suggesting a general property of compression-based learning systems.

Several directions remain open. Direct comparison with ILP systems (ILASP, Metagol) on shared benchmarks would position DreamLog precisely in the landscape of rule learners. Support for recursive rule discovery would address the ancestor predicate gap. Transfer learning experiments (training on one domain, compressing a combined KB with a second) would test whether compression discovers cross-domain abstractions. Scaling beyond 1000 clauses requires sampled verification. Finally, a richer MDL metric incorporating clause complexity (variable count, body length, nesting depth) would bring the system closer to Solomonoff's theoretical ideal.

Code and Data Availability

The DreamLog implementation, all experiment scripts (EX23 through EX26), the canonical and crafting domain data, the Popper input files used for the EX26 baseline, and the experiment registry recording the full provenance of each result are available at https://github.com/queelius/dreamlog. The EX26 Popper baseline was run against pinned commit `af39e522` of the Popper repository; the pinned hash is recorded in the registry. Anthropic Claude Haiku 4.5 is used for Operation G; total LLM cost across all reported experiments is under $0.10.

References

[1] R. J. Solomonoff, "A formal theory of inductive inference. part i," *Information and control*, vol. 7, no. 1, pp. 1–22, 1964.

[2] J. Rissanen, "Modeling by shortest data description," *Automatica*, vol. 14, no. 5, pp. 465–471, 1978.

[3] P. D. Grünwald, *The Minimum Description Length Principle*. MIT Press, 2007.

[4] K. Ellis, C. Wong, M. Nye, M. Sablé-Meyer, L. Morales, L. Hewitt, L. Cary, A. Solar-Lezama, and J. B. Tenenbaum, "Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning," in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2021, pp. 835–850.

[5] A. Cropper and R. Morel, "Learning programs by learning from failures," *Machine Learning*, vol. 110, no. 4, pp. 801–856, 2021.

[6] G. Delétang, A. Ruoss, P.-A. Duquenne, E. Catt, T. Genewein, C. Mattern, J. Grau-Moya, L. K. Wenliang, M. Aitchison, L. Orseau, M. Hutter, and J. Veness, "Language modeling is compression," in *International Conference on Learning Representations (ICLR)*, 2024.

[7] S. Lotfi, M. Finzi, S. Kapoor, A. Potapczynski, M. Goldblum, and A. G. Wilson, "PAC-Bayes compression bounds so tight that they can explain generalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[8] S. Dumancic, T. Guns, and A. Cropper, "Knowledge refactoring for inductive program synthesis," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 8, 2021, pp. 7271–7278, arXiv:2004.09931.

[9] A. Cropper, "Forgetting to learn logic programs," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 3676–3683, arXiv:1911.06643.

[10] R. Evans, J. Hernández-Orallo, J. Welbl, P. Kohli, and M. Sergot, "Making sense of sensory input," *Artificial Intelligence*, vol. 293, p. 103438, 2021, arXiv:1910.02227.

[11] K. Ellis, L. Wong, M. Nye, M. Sablé-Meyer, L. Cary, L. Anaya Pozo, L. Hewitt, A. Solar-Lezama, and J. B. Tenenbaum, "DreamCoder: growing generalizable, interpretable knowledge with wake–sleep Bayesian program learning," *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 381, no. 2251, p. 20220050, 2023.

[12] M. Bowers, T. X. Olausson, L. Wong, G. Grand, J. B. Tenenbaum, K. Ellis, and A. Solar-Lezama, "Top-down synthesis for library learning," *Proceedings of the ACM on Programming Languages*, vol. 7, no. POPL, pp. 1182–1213, 2023.

[13] G. Grand, L. Wong, M. Bowers, T. X. Olausson, M. Liu, J. B. Tenenbaum, and J. Andreas, "LILO: Learning interpretable libraries by compressing and documenting code," in *International Conference on Learning Representations (ICLR)*, 2024.

[14] S. Muggleton and L. De Raedt, "Inductive logic programming: Theory and methods," *The Journal of Logic Programming*, vol. 19, pp. 629–679, 1994.

[15] J. R. Quinlan, "Learning logical definitions from relations," *Machine learning*, vol. 5, no. 3, pp. 239–266, 1990.

[16] S. Muggleton, "Inverse entailment and progol," *New generation computing*, vol. 13, no. 3, pp. 245–286, 1995.

[17] S. H. Muggleton, D. Lin, N. Pahlavi, and A. Tamaddoni-Nezhad, "Meta-interpretive learning: application to grammatical inference," *Machine Learning*, vol. 94, no. 1, pp. 25–49, 2014.

[18] M. Law, A. Russo, and K. Broda, "ILASP: Learning answer set programs from examples," in *Proceedings of the 24th International Conference on Logic Programming*, 2014.

[19] R. Evans and E. Grefenstette, "Learning explanatory rules from noisy data," *Journal of Artificial Intelligence Research*, vol. 61, pp. 1–64, 2018.

[20] T. Rocktäschel and S. Riedel, "End-to-end differentiable proving," in *Advances in Neural Information Processing Systems*, 2017, pp. 3788–3800.

[21] T. X. Olausson, A. Gu, B. Lipkin, C. E. Zhang, A. Solar-Lezama, J. B. Tenenbaum, and R. Levy, "LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 5153–5176.

[22] G. D. Plotkin, "A note on inductive generalization," *Machine Intelligence*, vol. 5, pp. 153–163, 1970.

[23] J. C. Reynolds, "Transformational systems and the algebraic structure of atomic formulas," *Machine Intelligence*, vol. 5, pp. 135–151, 1970.

[24] J. A. Robinson, "A machine-oriented logic based on the resolution principle," *Journal of the ACM*, vol. 12, no. 1, pp. 23–41, 1965.

[25] I. Stahl, "Predicate invention in inductive logic programming," *Advances in Inductive Logic Programming*, pp. 34–47, 1995.

[26] S. Muggleton and W. Buntine, "Machine invention of first-order predicates by inverting resolution," in *Proceedings of the 5th International Conference on Machine Learning*, 1988, pp. 339–352.

[27] A. Cropper, S. Dumancic, R. Evans, and S. H. Muggleton, "Inductive logic programming at 30: a new introduction," *Journal of Artificial Intelligence Research*, vol. 74, pp. 765–850, 2022.

[28] R. Manhaeve, S. Dumancic, A. Kimmig, T. Demeester, and L. De Raedt, "DeepProbLog: Neural probabilistic logic programming," in *Advances in Neural Information Processing Systems*, 2018, pp. 3749–3759.

[29] L. Serafini and A. d'Avila Garcez, "Logic tensor networks: Deep learning and logical reasoning from data and knowledge," in *Neural-Symbolic Learning and Reasoning (NeSy)*, 2016.

[30] F. Yang, Z. Yang, and W. W. Cohen, "Differentiable learning of logical rules for knowledge base reasoning," in *Advances in Neural Information Processing Systems*, 2017, pp. 2319–2328.

[31] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner *et al.*, "Relational inductive biases, deep learning, and graph networks," *arXiv preprint arXiv:1806.01261*, 2018.

[32] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in *Advances in Neural Information Processing Systems*, 2013, pp. 2787–2795.
[33] Z. Sun, Z.-H. Deng, J.-Y. Nie, and J. Tang, "RotatE: Knowledge graph embedding by relational rotation in complex space," in *International Conference on Learning Representations*, 2019.
[34] L. Pan, A. Albalak, X. Wang, and W. Y. Wang, "Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.
[35] R. Wang, E. Zelikman, G. Poesia, Y. Pu, N. Haber, and N. D. Goodman, "Hypothesis search: Inductive reasoning with language models," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2309.05660.
[36] L. Qiu, L. Jiang, X. Lu, M. Sclar, V. Pyatkin, C. Bhagavatula, B. Wang, Y. Kim, Y. Choi, N. Dziri, and X. Ren, "Phenomenal yet puzzling: Testing inductive reasoning capabilities of language models with hypothesis refinement," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2310.08559.
[37] Z. Zhu, Y. Xue, X. Chen, D. Zhou, J. Tang, D. Schuurmans, and H. Dai, "Large language models can learn rules," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2310.07064.
[38] J. P. G. de Souza, D. Carvalho, and A. Freitas, "Inductive learning of logical theories with LLMs: An expressivity-graded analysis," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 22, 2025, pp. 23 752–23 759, arXiv:2408.16779.
[39] K. He and Z. Chen, "From reasoning to learning: A survey on hypothesis discovery and rule learning with large language models," *arXiv preprint arXiv:2505.21935*, 2025.
[40] T. Mitchell, W. Cohen, E. Hruschka, P. Talukdar, B. Yang, J. Betteridge, A. Carlson, B. Dalvi, M. Gardner, B. Kisiel *et al.*, "Never-ending learning," *Communications of the ACM*, vol. 61, no. 5, pp. 103–115, 2018.
[41] J. W. Lloyd, *Foundations of Logic Programming*, 2nd ed. Springer, 1987.
[42] K. L. Clark, "Negation as failure," *Logic and Data Bases*, pp. 293–322, 1978.